

Matrices as Linear Maps

A matrix is a function that moves space

Prof. Nipun Batra

Mathematical Foundations for AI · IIT Gandhinagar

Matrices as transformations in neural networks

Rotate a photo: one matrix moves every pixel

original image (a grid of pixels)



every pixel $\times$ a rotation matrix



every pixel $\times$ a shear matrix



Coordinate map versus rendered image

The geometric move starts with one 2×2 matrix applied to pixel coordinates.

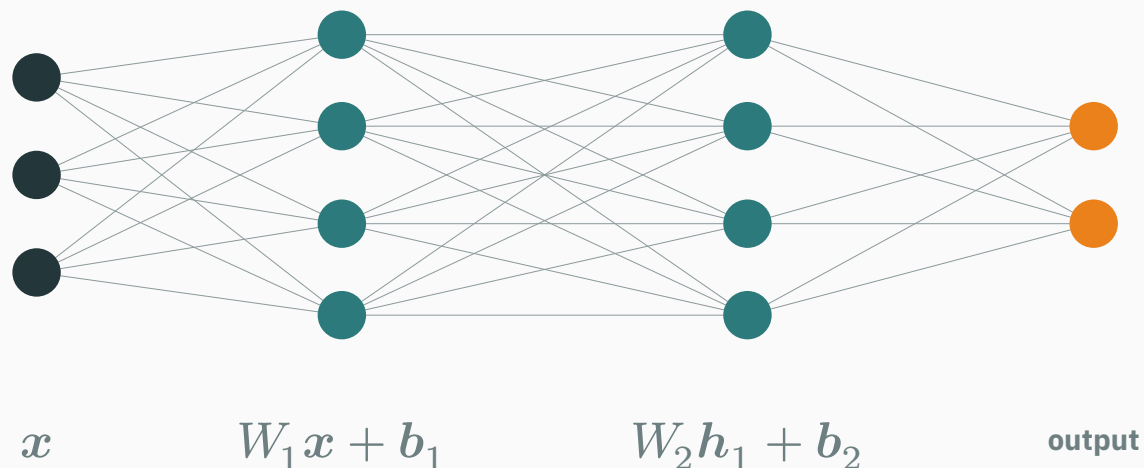
Producing a usable output image needs two additional numerical steps:

- map each output pixel back to a source coordinate so there are no holes,
- interpolate when that coordinate falls between source pixels.

Change four matrix entries and the coordinate move changes from rotation to shear, zoom, or reflection. Today we study that map; image resampling is a separate numerical problem.

Inside a dense neural-network layer

A dense layer computes $\mathbf{h} = \varphi(W\mathbf{x} + \mathbf{b})$. Attention, normalization, and residual connections add other operations, but matrix maps remain a central primitive.



The learned affine map $W\mathbf{x} + \mathbf{b}$ moves and translates space; the fixed nonlinearity φ bends it. Both W and $\mathbf{b}$ are trained. Today we isolate the matrix part: $W\mathbf{x}$.

What a dense-layer picture leaves out

A GPT-class model also uses:

- attention to mix information across token positions,
- normalization and residual additions to control signal flow,
- nonlinearities and output-specific operations.

All of these interact with large matrix products. Understanding a single matrix map is the linear-algebra starting point, not a description of every layer.

Learning outcomes

By the end of this lecture you will be able to:

1. Read a small matrix as a **move of space** — and sketch what it does to a grid.
2. Compute Ax two ways: the **row view** (dot products) and the **column view** (mix of columns).
3. Explain why matrix $\times$ matrix is **composition** of moves — and why $AB \neq BA$.
4. Compute **rank** on small matrices and say what it means geometrically: how much of space survives.
5. Decide when a matrix is **invertible** — and recognize when information is destroyed for good.
6. Solve $Ax = b$ numerically with `solve` rather than forming an explicit inverse, and set up **least squares** for tall systems.

A matrix is a function

A second identity for an old object

So far a matrix has been a **table**: in L3, rows were data points (n students $\times d$ features).

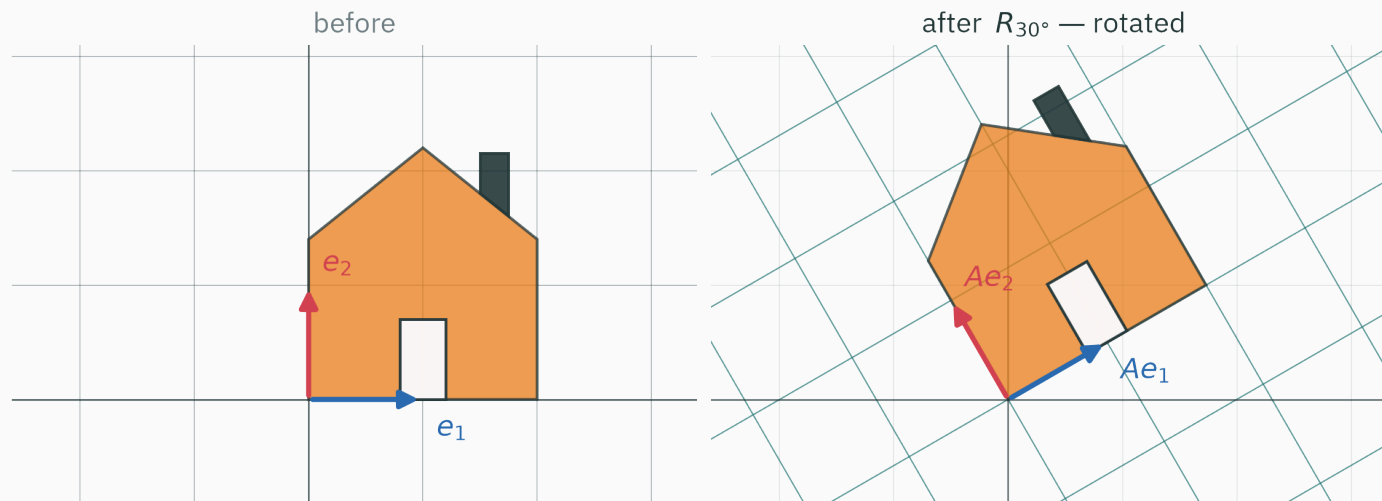
Today the same object gets a second life. Feed a 2×2 matrix a vector — it hands back a vector:

$$T(\mathbf{x}) = A\mathbf{x}, \quad T : \mathbb{R}^2 \rightarrow \mathbb{R}^2$$

That makes A a **function**. Its input is every point of the plane; its output is where each point moves.

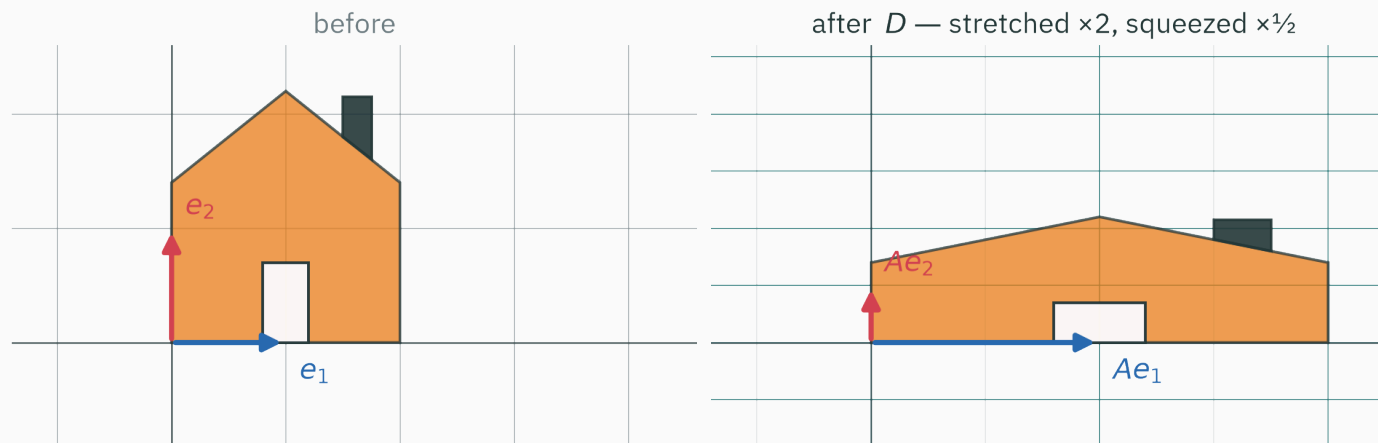
Same numbers, two readings — the table stores measurements; the function **does** something. Keep both in your head: ML constantly switches between them mid-equation.

Example 1 · rotation



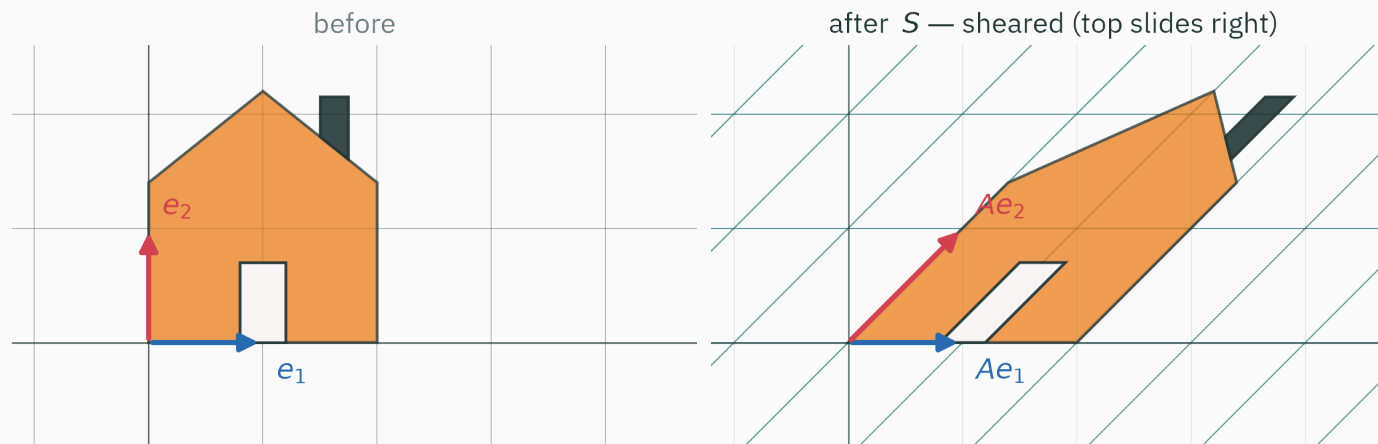
$$R_{30^\circ} = \begin{pmatrix} \cos 30^\circ & -\sin 30^\circ \\ \sin 30^\circ & \cos 30^\circ \end{pmatrix} \approx \begin{pmatrix} 0.87 & -0.50 \\ 0.50 & 0.87 \end{pmatrix} \quad \text{— the whole plane turns; the house rides along}$$

Example 2 · stretch and squeeze



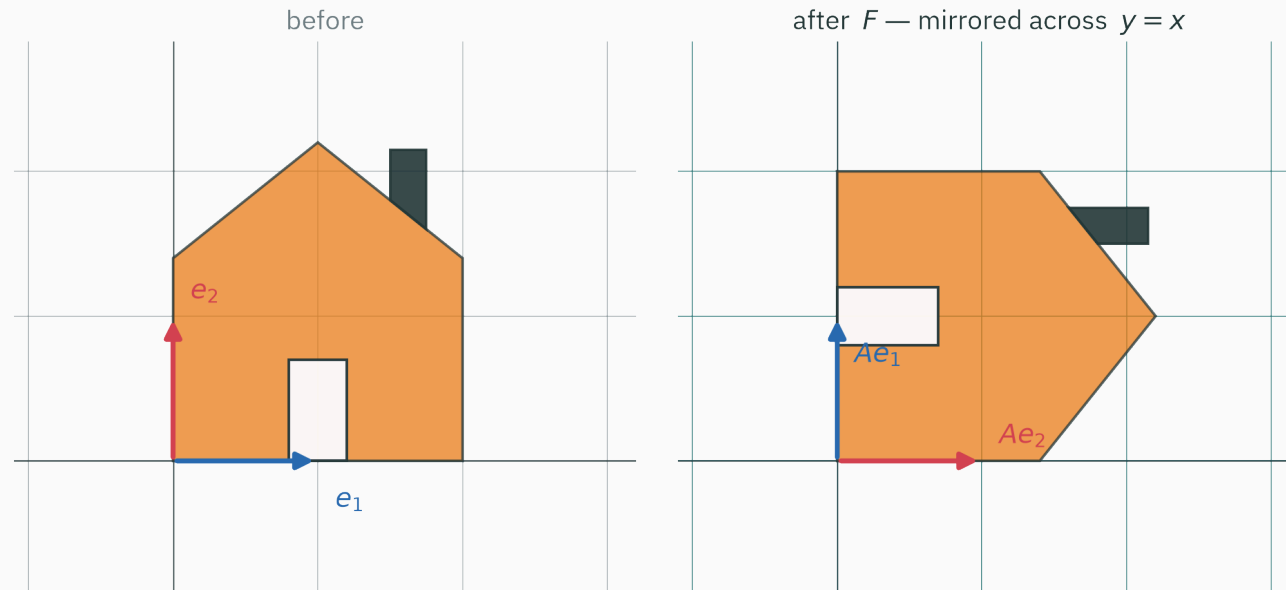
$D = \begin{pmatrix} 2 & 0 \\ 0 & 1/2 \end{pmatrix}$ — $\times 2$ along x , $\times 1/2$ along y ; the grid squares become bricks

Example 3 · shear



$$S = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \quad \text{— the ground floor holds still; every higher layer slides right}$$

Example 4 · reflection



$$F = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \text{ — mirror across the line } y = x$$

The chimney switched sides — no rotation can ever do that. Four matrices, four geometrically different moves.

What all four examples preserved

Look back at the after-grids. Every move kept three promises:

- straight lines stayed **straight** (nothing bent)
- parallel lines stayed **parallel**, evenly spaced — grid squares became identical tilted bricks
- the **origin stayed put**

Moves that keep these promises are called **linear**, and they obey two laws:

$$A(u + v) = Au + Av, \quad A(cu) = c(Au)$$

Linear = the move respects addition and scaling. Every claim in today's lecture flows from these two laws.

The mechanics · one entry at a time

How is Ax actually computed? Each output entry is a **row of A dotted with x** :

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} ax_1 + bx_2 \\ cx_1 + dx_2 \end{pmatrix}$$

Check it on the shear, moving the point $(2, 1)$:

$$\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 + 1 \\ 0 + 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 1 \end{pmatrix} \quad \checkmark \text{ slid right, height kept}$$

L3 callback: each output entry is a **dot product** — row i asks the input “how much of you lies along me?” A neural layer with 4096 rows asks 4096 such questions at once.

The shape contract, in color

Matrices multiply vectors of **matching** shape — dimensions in **orange** must agree, and they are consumed:

$$\underbrace{\begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \end{pmatrix}}_{2 \times 3} \underbrace{\begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix}}_{3 \times 1} = \underbrace{\begin{pmatrix} 1 \cdot 1 + 2 \cdot 2 + 0 \cdot 1 \\ 0 \cdot 1 + 1 \cdot 2 + 3 \cdot 1 \end{pmatrix}}_{2 \times 1} = \begin{pmatrix} 5 \\ 5 \end{pmatrix}$$

The general contract — inner dimensions must match and are consumed; outer dimensions survive:

$$(m \times n) \cdot (n \times 1) \rightarrow (m \times 1)$$

An $m \times n$ matrix eats vectors from $\mathbb{R}^n$ and returns vectors in $\mathbb{R}^m$ — it can move you **between spaces.**

Code · one @ moves a million points

```
import numpy as np
th = np.deg2rad(30)
R = np.array([[np.cos(th), -np.sin(th)],
              [np.sin(th),  np.cos(th)]])

R @ np.array([2.0, 1.0])      # array([1.232, 1.866]) – one point moved
```

```
pts = np.random.rand(1_000_000, 2)  # a million pixel coordinates
new = pts @ R.T                      # ...all rotated in one line
```

That second line **is** the photo app from the opening slide. No loops — the matrix moves all of space at once, and hardware loves it.

Symbols · what “linear map” means

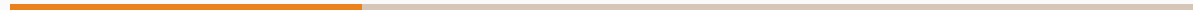
A map $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is **linear** if for all vectors u, v and scalars c :

$$T(u + v) = T(u) + T(v), \quad T(cu) = cT(u)$$

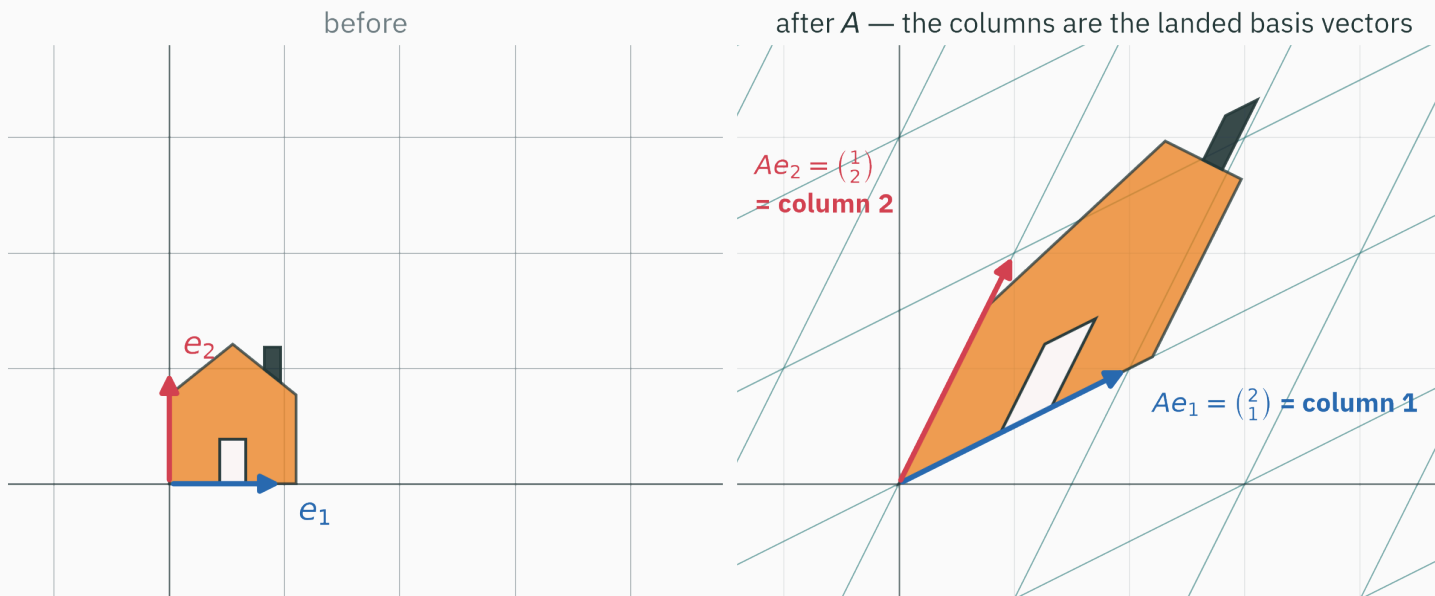
- Every matrix gives a linear map $x \mapsto Ax$ — the two laws are exactly how the row-dot-product formula behaves.
- The converse holds too (stated, not proved): **every** linear map $\mathbb{R}^n \rightarrow \mathbb{R}^m$ is multiplication by some $m \times n$ matrix. There is nothing else.

Every matrix is a linear map; every linear map $\mathbb{R}^n \rightarrow \mathbb{R}^m$ is a matrix. One concept, two notations.

Read the columns



Watch just two vectors



You do not need to track the whole plane: the two basis-vector images determine the map.

The columns are the landed basis

Where does $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ send the basis? Just multiply:

$$Ae_1 = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \text{column 1}, \quad Ae_2 = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \text{column 2}$$

This is completely general — feeding in e_j picks out column j :

Ae_j = the j -th column of A

Sanity check with the do-nothing move: the identity $I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ has columns e_1, e_2 — “the basis lands exactly where it started.” $Ix = x$ for every x .

Why do two arrows determine everything? Any input is a mix of basis vectors:

$$x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = x_1 e_1 + x_2 e_2$$

Now apply A and let linearity do all the work:

$$Ax = A(x_1 e_1 + x_2 e_2) = x_1 (Ae_1) + x_2 (Ae_2)$$

But Ae_1, Ae_2 are the **columns** a_1, a_2 :

$$Ax = x_1 a_1 + x_2 a_2$$

Ax is a recipe: mix the columns of A , using the entries of x as amounts.
(Strang's column picture.)

Numbers · two readings, one answer

Compute Ax for $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$, $x = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$ — both ways:

Row view — two dot products:

$$\begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \cdot 2 + 1 \cdot 1 \\ 1 \cdot 2 + 2 \cdot 1 \end{pmatrix} = \begin{pmatrix} 5 \\ 4 \end{pmatrix}$$

Column view — a mix of columns, 2 parts and 1 part:

$$2 \begin{pmatrix} 2 \\ 1 \end{pmatrix} + 1 \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} 4 \\ 2 \end{pmatrix} + \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} 5 \\ 4 \end{pmatrix} \quad \checkmark \text{ same answer}$$

The row view is how machines compute. The column view is how **you should think** — it will explain rank, $Ax = b$, and least squares within the hour.

Construct a matrix for a desired transformation

Want a matrix that rotates by 90° ? Don't memorize — **place the columns**:

e_1 must land at $(0, 1)$; e_2 at $(-1, 0)$. The landings **are** the columns:

$$R_{90^\circ} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$$

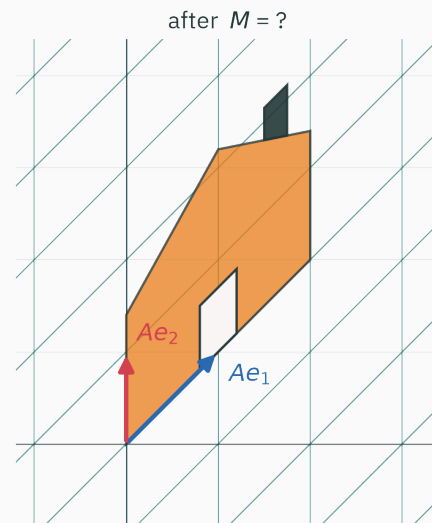
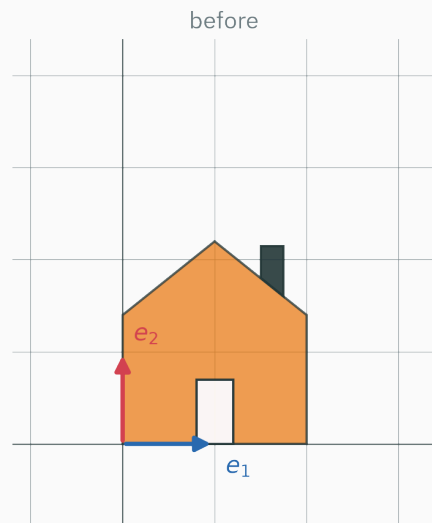
A “shadow” matrix that flattens everything onto the x -axis? e_1 stays, e_2 dies:

$$P = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$$

That second design **crushed a whole dimension on purpose** — P is our first **rank-1** matrix. Rank gets its definition in the next section.

Checkpoint: identify the transformation

QUESTION



Which matrix M made this move?

A. $M = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$

B. $M = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$

C. $M = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$

D. $M = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}$

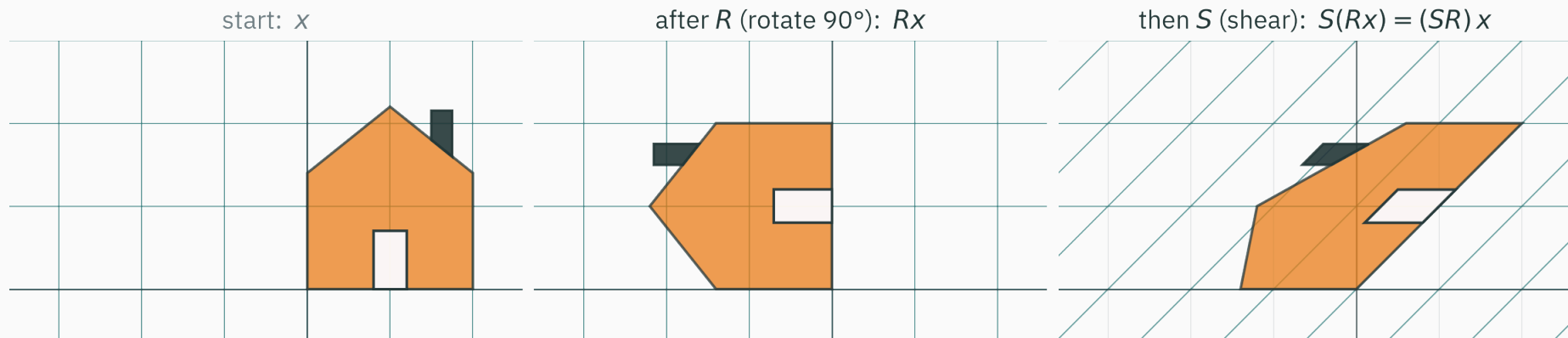
Correct: B — $M = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$

Why: Read the landings off the picture: e_1 landed at $(1, 1)$ — that must be column 1. e_2 stayed at $(0, 1)$ — that must be column 2. Only $\begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$ has those columns. (Option A is the **horizontal** shear from Example 3 — it would leave e_1 untouched instead.)

Every “which matrix did this?” puzzle is solved the same way: **watch where the basis lands**.

Multiplication is composition

Two moves, one after the other



Rotate with $R = R_{90^\circ} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$ (the rotation we built by columns), **then** shear with $S = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$ (Example 3): the input x becomes $S(Rx)$.

Two linear moves in a row are still one linear move — so **some single matrix** must do the whole trip. We name it SR , written right-to-left: the move nearest x acts first.

The combined matrix, without any formula

Where does the **combined** move send the basis? Just follow each arrow through both moves:

$$(SR)e_1 = S(R e_1) = S \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad (SR)e_2 = S(R e_2) = S \begin{pmatrix} -1 \\ 0 \end{pmatrix} = \begin{pmatrix} -1 \\ 0 \end{pmatrix}$$

The landings are the columns — the product is already done:

$$SR = \begin{pmatrix} 1 & -1 \\ 1 & 0 \end{pmatrix}$$

To multiply matrices, push the columns of the right one through the left one: column j of AB is Ab_j .

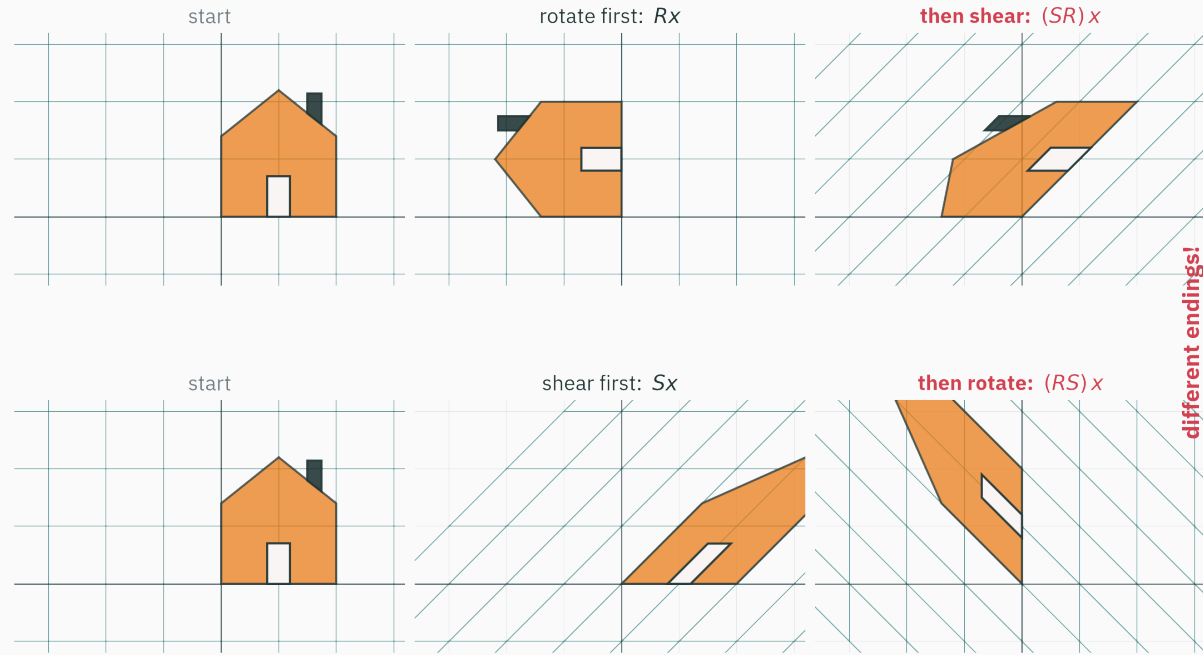
The general recipe (and its shape)

$$(AB)_{ij} = \sum_k A_{ik} B_{kj} \quad \text{— row } i \text{ of } A \text{ dotted with column } j \text{ of } B$$

The shape contract again — inner dimensions consumed, outer survive:

$$(m \times n) \cdot (n \times p) \rightarrow (m \times p)$$

- The entry formula is the **bookkeeping** view — what hardware executes.
- Column-through view is the **thinking** view: $AB = “A \text{ applied to each column of } B”$ — a matmul is just p matvecs standing side by side.



Rotate-then-shear and shear-then-rotate end in **different places** — so $SR \neq RS$ before we compute a thing.

SR vs RS · the numbers agree with the picture

Track the basis both ways (or grind the entry formula — same result):

$$SR = \begin{pmatrix} 1 & -1 \\ 1 & 0 \end{pmatrix}, \quad RS = \begin{pmatrix} 0 & -1 \\ 1 & 1 \end{pmatrix}$$

Different matrices — matching the two different final houses in the figure.

$AB \neq BA$ in general. Matrix multiplication remembers the **order of moves** — “put on socks, then shoes” is not “put on shoes, then socks.” Every identity you write this semester must respect it.

What **does** survive from ordinary algebra: associativity, $(AB)C = A(BC)$ — grouping is free, order is not.

INTERACTIVE

↗ <http://matrixmultiplication.xyz/>

Enter $S = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$ and $R = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$ and watch the row-through-column machine grind out each entry — the row physically tips over and slides down the column. In class: predict every entry of RS **before** the animation lands.

Then feed it a $(2 \times 3)(3 \times 1)$ pair and watch the inner 3s consume each other — the shape contract, animated.

Code · @ composes

```
S = np.array([[1., 1.], [0., 1.]])
R = np.array([[0., -1.], [1., 0.]])

S @ R          # array([[ 1., -1.], [ 1.,  0.]])
R @ S          # array([[ 0., -1.], [ 1.,  1.]]) - different!

x = np.array([2., 1.])
np.allclose((S @ R) @ x, S @ (R @ x))    # True - associativity
```

That last line matters in practice: **precompute the combined move once**, or apply moves one at a time — identical result. Graphics engines and deep-learning compilers rely on this choice.

Why stacked linear layers need a nonlinearity

Stack two purely linear layers, $h = W_1 x$ then $y = W_2 h$:

$$y = W_2(W_1 x) = (W_2 W_1)x = W_{\text{eq}} x$$

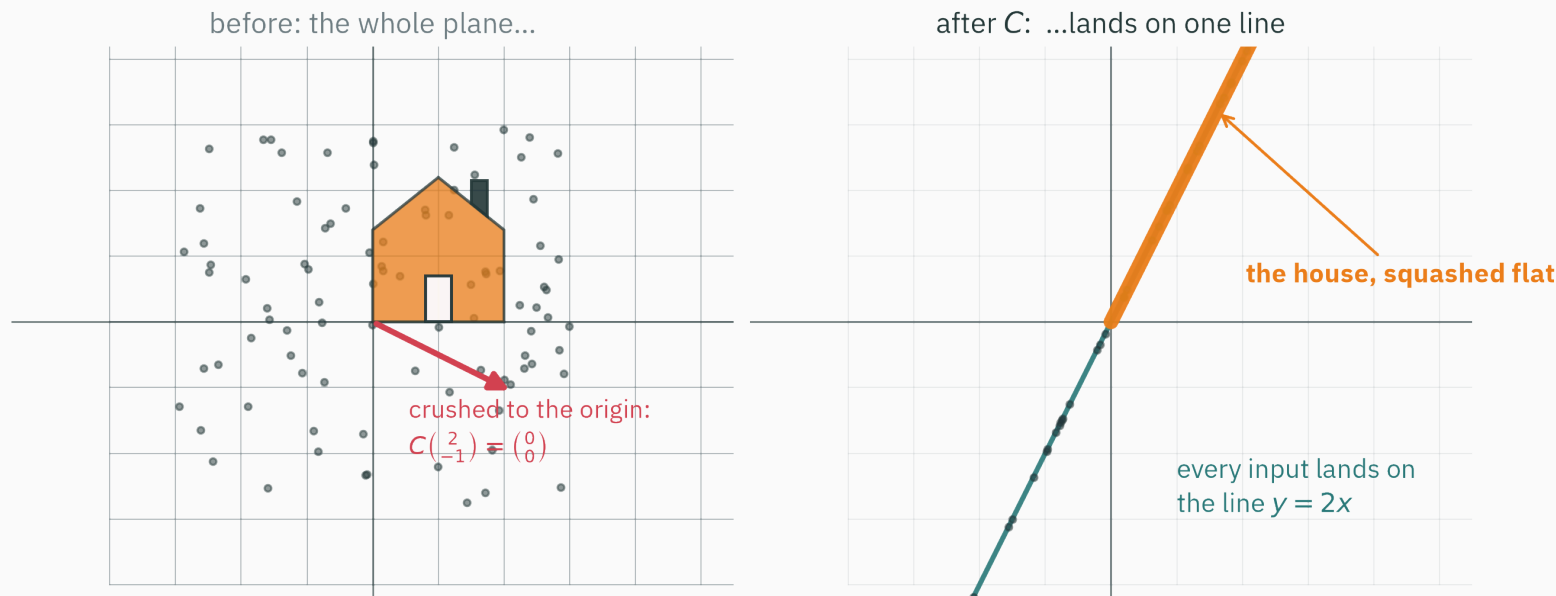
Composition collapses the stack: a 100-layer **linear** network is secretly **one matrix** — depth bought nothing.

A stack containing only affine maps still collapses to one affine map. Inserting nonlinearities prevents that algebraic collapse and lets depth represent non-affine functions.

Move of space → bend → move of space → bend is the core pattern of a multilayer perceptron. Other architectures add further operations; the matrix maps are today's mathematics.

**Rank: how much of space
survives**

Not every move preserves the plane



$C = \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$ takes the **entire 2-D plane** — houses, point clouds, everything — and lands it all on a single line. Something fundamental was lost.

The collapse, in numbers

Watch the columns of $C = \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$: column 2 is exactly $2 \times$ column 1.

So every output — every mix $x_1 \mathbf{a}_1 + x_2 \mathbf{a}_2$ — is a multiple of $\begin{pmatrix} 1 \\ 2 \end{pmatrix}$:

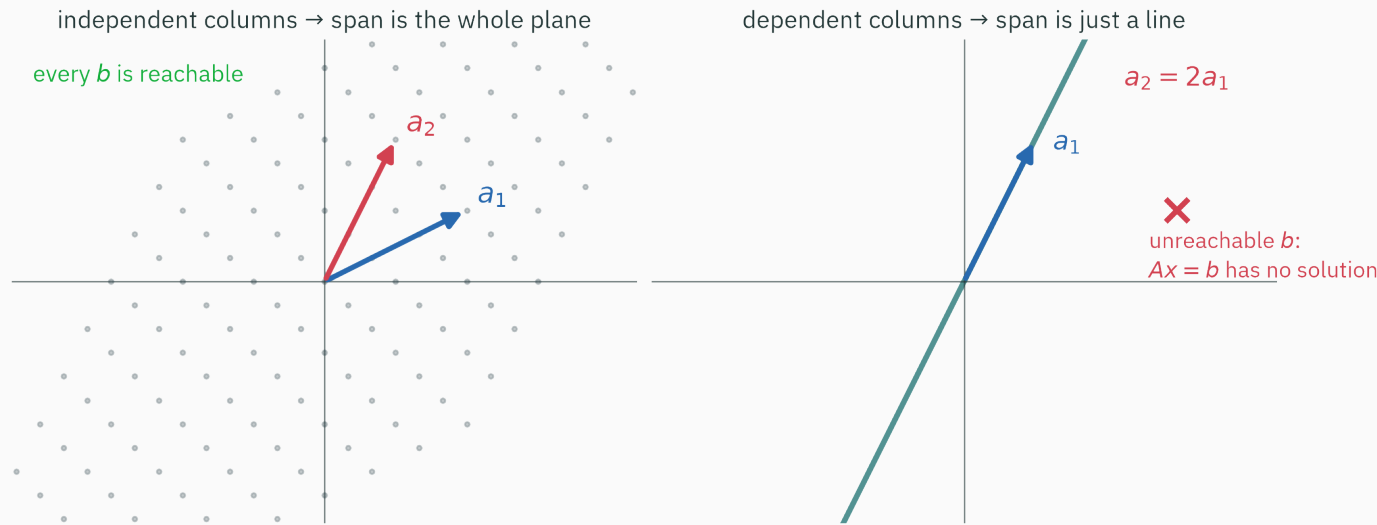
$$C\mathbf{x} = x_1 \begin{pmatrix} 1 \\ 2 \end{pmatrix} + x_2 \begin{pmatrix} 2 \\ 4 \end{pmatrix} = (x_1 + 2x_2) \begin{pmatrix} 1 \\ 2 \end{pmatrix} \quad \text{— one direction, scaled. The line } y = 2x.$$

And one whole direction of inputs is sent to the origin:

$$C \begin{pmatrix} 2 \\ -1 \end{pmatrix} = 2 \begin{pmatrix} 1 \\ 2 \end{pmatrix} - 1 \begin{pmatrix} 2 \\ 4 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

Both columns point along the same direction, so one whole input dimension is lost.

What the columns can reach · column space



$\text{col}(A) = \text{span}\{a_1, \dots, a_n\}$ — L3's **span** of the columns: everything Ax can ever reach.

Dependent columns shrink the span to a line, and most targets b become **unreachable** — exactly when $Ax = b$ has no solution. (We return to $Ax = b$ in the final section.)

Rank · the dimension of what survives

$\text{rank}(A) = \text{number of linearly independent columns} = \dim(\text{col}(A))$

Move	Matrix	Rank	What survives
rotation R_{30°	$\begin{pmatrix} 0.87 & -0.5 \\ 0.5 & 0.87 \end{pmatrix}$	2	the whole plane
shear S	$\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$	2	the whole plane
shadow P	$\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$	1	the x -axis
collapse C	$\begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$	1	the line $y = 2x$
zero matrix O	$\begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$	0	just the origin

For any $m \times n$ matrix: $\text{rank}(A) \leq \min(m, n)$ — you can't have more independent columns than columns, nor than the dimension they live in. Hitting the ceiling is called **full rank**.

The null space · what dies

The flip side of “what survives” is **which inputs get crushed to zero**:

$$\text{null}(A) = \{\mathbf{x} : A\mathbf{x} = \mathbf{0}\} \quad \text{for } C : \text{the whole line through } \begin{pmatrix} 2 \\ -1 \end{pmatrix}$$

Dimensions are conserved — every input dimension either survives or dies:

$$\underbrace{\text{rank}(A)}_{\text{survives}} + \underbrace{\dim \text{null}(A)}_{\text{dies}} = \underbrace{n}_{\text{input dimensions}} \quad \text{for } C : 1 + 1 = 2 \checkmark$$

(The **rank–nullity theorem** — stated here, proved in MML 2.7.)

L5 preview: a direction that A merely **scales** — stretches without turning — is called an **eigenvector**. A direction crushed to zero is the extreme case: an eigenvector with stretch factor 0. Next lecture is entirely about these special directions.

Rank by hand, one size up

$$A = \begin{pmatrix} 0 & 1 & 2 \\ 1 & 2 & 1 \\ 2 & 7 & 8 \end{pmatrix} \quad \text{— } 3 \times 3, \text{ so rank} \leq 3. \text{ Hunt for a dependency.}$$

Row 3 is a mix of the others: $3 \cdot \text{row 1} + 2 \cdot \text{row 2} = (2, 7, 8) = \text{row 3}$. Rows 1, 2 are not proportional, so exactly 2 independent rows $\rightarrow \text{rank}(A) = 2$.

Row rank always equals column rank (MML 2.6), so test dependencies on whichever side is easier. Here the row dependency is immediate.

One more: $X = \begin{pmatrix} 1 & 2 & 4 & 4 \\ 3 & 4 & 8 & 0 \end{pmatrix}$ is 2×4 , so $\text{rank} \leq 2$; its two rows aren't proportional $\rightarrow \text{rank}(X) = 2$ — full rank, even though column 3 is $2 \times$ column 2.

The thinnest matrices · outer products

Take $u = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$ and $v = \begin{pmatrix} 4 \\ 5 \\ 6 \end{pmatrix}$. Their **outer product** $uv^\top$ is a full 3×3 matrix:

$$uv^\top = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} (4 \ 5 \ 6) = \begin{pmatrix} 4 & 5 & 6 \\ 8 & 10 & 12 \\ 12 & 15 & 18 \end{pmatrix}$$

Cell (i, j) is simply $u_i v_j$ — a multiplication table.

	$v_1 = 4 \ v_2 = 5 \ v_3 = 6$		
$u_1 = 1$	4	5	6
$u_2 = 2$	8	10	12
$u_3 = 3$	12	15	18

Nine entries, but look at the shading: every column repeats the **same pattern**, just rescaled. Nine numbers carrying only one column's worth of independent information.

Claim: for any nonzero u, v , the outer product $uv^\top$ has rank exactly 1.

Look at the columns. Column j of $uv^\top$ is $v_j u$ — every column is a multiple of the **same** vector:

$$\text{col } 1 = 4u, \quad \text{col } 2 = 5u, \quad \text{col } 3 = 6u$$

So the column space is just the line through u : dimension 1 $\rightarrow$ rank 1. ■

Second look, via the action: $(uv^\top)x = u(v^\top x)$ — a **number** times u . Every input in $\mathbb{R}^3$ lands on u 's line: the map measures you against v , then points along u .

An outer product $uv^\top$ is the thinnest nonzero matrix there is: rank 1.

Every matrix is a sum of rank-1 atoms

Read the claim backwards: rank-1 outer products are **atoms**, and every matrix is a molecule —

$$A = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^\top + \sigma_2 \mathbf{u}_2 \mathbf{v}_2^\top + \cdots + \sigma_r \mathbf{u}_r \mathbf{v}_r^\top, \quad r = \text{rank}(A)$$

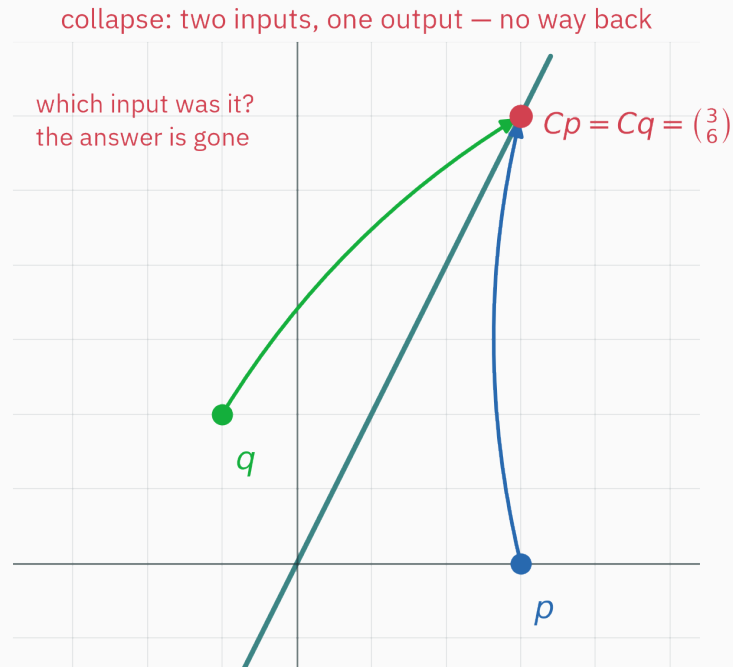
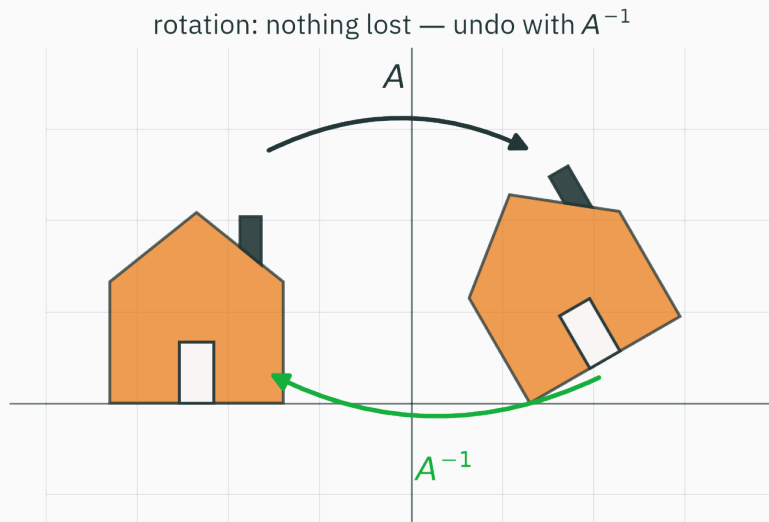
for suitable vectors $\mathbf{u}_i, \mathbf{v}_i$ and weights $\sigma_i > 0$ (stated here; L6 constructs them).

- L6 preview: the **SVD** finds the best atoms, importance-sorted $\sigma_1 \geq \sigma_2 \geq \dots$ — keep the top few → compression, PCA.
- Modern fine-tuning (**LoRA**) freezes a giant W and learns only a low-rank update — a few outer products instead of billions of entries.

The statement “every column is a multiple of $\mathbf{u}$ ” is the rank-one structure used by low-rank parameter updates in L6.

Invertibility: undoing a move

Can you undo the move?



The rotation is **reversible** — every output point has exactly one input. The collapse is not: p and q land on the **same** output, so the input cannot be recovered uniquely.

The inverse · undo, in numbers

A^{-1} is the move that cancels A exactly — do, then undo, and nothing happened:

$$A^{-1}A = AA^{-1} = I$$

For our exhibits the undo is guessable from the **geometry**, no formula needed:

$$R_{30^\circ}^{-1} = R_{-30^\circ} \quad (\text{rotate back}), \quad S^{-1} = \begin{pmatrix} 1 & -1 \\ 0 & 1 \end{pmatrix} \quad (\text{slide the top back left})$$

Verify the shear by composing:

$$\begin{pmatrix} 1 & -1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I \quad \checkmark$$

When an inverse exists, it is **unique** — there is only one way to perfectly undo a move.

When can you undo? Three equivalent tests

For a **square** matrix A , these are the same single question:

- **full rank** — columns independent: the outputs fill all of $\mathbb{R}^n$, nothing was flattened
- **null space is only $\{0\}$** — no nonzero direction is removed, so distinct inputs do not collide
- $\det(A) \neq 0$ — the determinant measures the **area scale factor** of the move; $\det = 0$ means areas got crushed flat (we'll meet $\det$ properly in L5)

A matrix passing the test is **invertible** (nonsingular); failing, **singular**.

Invertible $\Leftrightarrow$ full rank $\Leftrightarrow$ nothing is crushed. You can undo a move exactly when it destroyed no information.

Without computing a single determinant: which matrix has **no inverse?
(Hint: one of them flattens the plane.)**

A. $\begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$

B. $\begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$

C. $\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$

D. $\begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$

Correct: B — $\begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$ is singular

Why: Its second column is $2 \times$ the first — dependent columns $\rightarrow$ rank 1 $\rightarrow$ the plane is flattened onto a line $\rightarrow$ two different inputs share each output, so no undo can exist. The rotation (A), shear (C), and scale (D) all keep rank 2: nothing dies, so each can be undone.

Notice you **never** needed arithmetic — reading the columns answered a determinant question.

Solving $Ax = b$

The most-solved equation in computing

Flip the question. So far: given x , where does it land? Now **the landing b is known; the input is not:**

$Ax = b$ — which input lands on b ?

The column picture makes it concrete — **find the recipe** mixing the columns into b :

$$\begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} x = \begin{pmatrix} 4 \\ 5 \end{pmatrix} \Rightarrow x = \begin{pmatrix} 1 \\ 2 \end{pmatrix}: \quad 1 \begin{pmatrix} 2 \\ 1 \end{pmatrix} + 2 \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} 4 \\ 5 \end{pmatrix} \quad \checkmark$$

Solvable for every b , uniquely, exactly when A is invertible — nothing crushed. By hand for two unknowns; but one weather step or network layer has $n \approx 10^6$. We need the machine — **carefully**.

On paper, $x = A^{-1}b$ · in code, use `solve`

$x = A^{-1}b$ is a theorem, **not an algorithm**. Watch both routes on a notoriously nasty (but tiny!) matrix — the 12×12 Hilbert matrix of simple fractions $H_{ij} = 1/(i + j + 1)$, indices from 0 as in NumPy, with true solution all-ones:

```
n = 12; i = np.arange(n)
H = 1.0 / (i[:, None] + i[None, :] + 1)      # Hilbert matrix
x_true = np.ones(n); b = H @ x_true

err_inv = np.abs(np.linalg.inv(H) @ b - x_true).max()
err_solve = np.abs(np.linalg.solve(H, b) - x_true).max()
print(err_inv, err_solve)      # here: 8.66 vs 0.14
```

The true entries are all 1. Both routes struggle because H is extremely ill-conditioned — but on this machine `inv` misses by 8.66 where `solve` misses by 0.14, roughly $60 \times$ worse. Exact values vary across NumPy/BLAS builds; the ordering is typical.

Why solve wins

- **Fewer operations** — `inv` computes solutions for all n columns of I , then still multiplies by b . If one only needs $Ax = b$, that is unnecessary work.
- **Better numerical route** — `solve` factors A and applies triangular solves directly. Forming an approximate A^{-1} and multiplying introduces extra work and another source of rounding error.

When the task is to solve $Ax = b$, write `np.linalg.solve(A, b)`. Explicit inverses have legitimate uses, but this is normally not one of them.

Honest footnote: $\text{cond}(H) \approx 10^{16}$, so even `solve` lost most of its digits here (error 0.14). Why some matrices **amplify** rounding error — the condition number — is where Lecture 17 begins.

Too many equations · tall systems

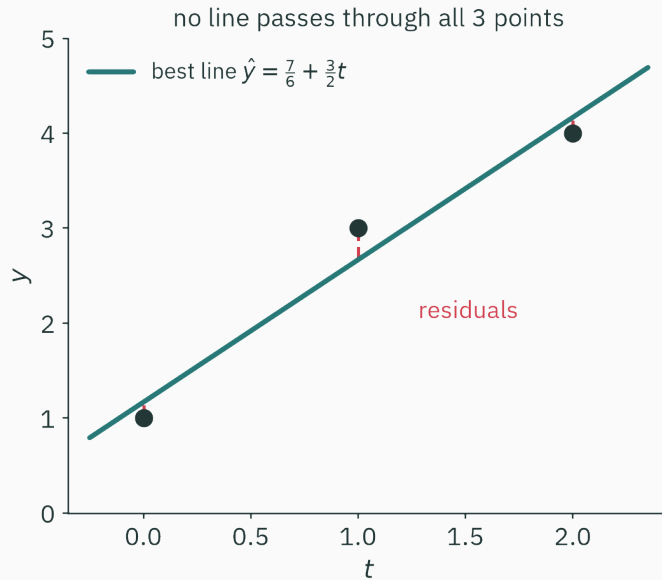
Fit a line $\hat{y} = a + bt$ through **three** points $(0, 1), (1, 3), (2, 4)$ — three equations, two unknowns:

$$\underbrace{\begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{pmatrix}}_{3 \times 2 \text{ — tall}} \underbrace{\begin{pmatrix} a \\ b \end{pmatrix}}_{b \in \mathbb{R}^2} = \begin{pmatrix} 1 \\ 3 \\ 4 \end{pmatrix}$$

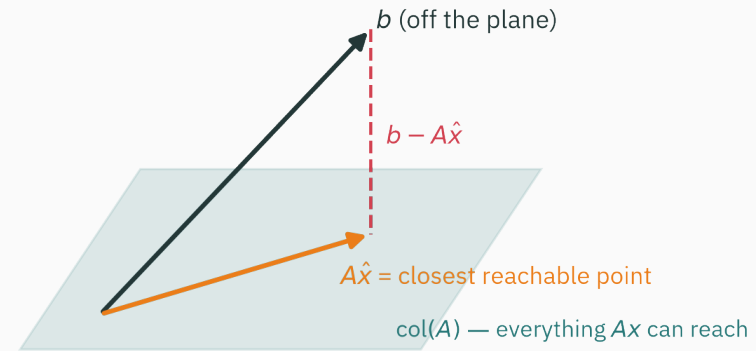
Now the column picture delivers bad news: $\text{col}(A)$ — everything the two columns can mix — is only a **plane** inside $\mathbb{R}^3$, and our b is off it.

A tall system generically has no exact solution: three demands, two dials.

No line fits · the miss, drawn



the same picture in column-space language



Left: no line threads all three points — every candidate leaves **residuals**. Right: the same fact in column-space language — b hovers off the reachable plane.

Least squares · aim for the shadow

If you can't reach b , reach the **closest point you can** — minimize the miss:

$$\hat{x} = \arg \min_x \|Ax - b\|^2$$

Geometrically (right panel, a moment ago): $A\hat{x}$ is the **projection** — L3's shadow! — of b onto $\text{col}(A)$; the leftover error sticks out perpendicular.

```
A = np.array([[1., 0.], [1., 1.], [1., 2.]])
b = np.array([1., 3., 4.])
np.linalg.lstsq(A, b, rcond=None)[0]    # (7/6, 3/2): best line 7/6 + 1.5·t
```

Two pointers ahead: **L6** computes least squares at scale (via SVD), and **L14** shows why **squared** error is the statistically right miss to minimize. Fitting = projection is the bridge.

Lecture 4 — summary

- **A matrix is a function**: $x \mapsto Ax$ moves all of space — lines stay straight, parallels parallel, origin fixed.
- **Columns = landed basis**: $Ae_j = a_j$, so $Ax = x_1 a_1 + \dots + x_n a_n$ — a recipe mixing columns.
- **Multiplication is composition**: column j of AB is Ab_j ; order matters, grouping doesn't.
- **Rank** = dimension of what survives; **null space** = what dies; $\text{rank} + \dim \text{null} = n$; $uv^\top$ = the rank-1 atom.
- **Invertible** $\Leftrightarrow$ **full rank**: undo exists iff no information was destroyed.
- For $Ax = b$, prefer **solve to inv(A) @ b** — less work and usually better numerical behaviour (conditioning: L17).
- **Tall systems**: no exact solution — least squares projects b onto $\text{col}(A)$ (L6, L14).

Before L5 and Tutorial 2

- Read MML Ch. 2.5–2.8.
- Revisit the transformation gallery and predict each matrix from its basis-vector images.
- In NumPy, experiment with rank and compare `solve` with `inv(A) @ b` on the Hilbert system; expect platform-dependent errors.

Practice problems · I

Try on paper; verify in the Tutorial 2 notebook.

1. Design by columns: the matrix reflecting across the x -axis, and the one rotating by 180° . Check by tracking e_1, e_2 .
2. Compute $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ and $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ — one swaps columns, one swaps rows. Which is which, and why?
3. Ranks of $\begin{pmatrix} 1 & 2 & 4 & 4 \\ 3 & 4 & 8 & 0 \end{pmatrix}$ and $\begin{pmatrix} 5 & 4 & 3 \\ 4 & 3 & 2 \\ 3 & 2 & 1 \end{pmatrix}$? Hunt for dependencies before trusting your gut.

Practice problems · II

4. For $u = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$, $v = \begin{pmatrix} 3 \\ 1 \end{pmatrix}$: write $uv^\top$ and find the direction it kills. Hint: what is perpendicular to v ?
5. Socks and shoes: why is $(AB)^{-1} = B^{-1}A^{-1}$? Think “undo two moves in the right order” — then verify on SR .
6. On the $n = 8$ Hilbert matrix, predict the more accurate of `solve` and `inv @ b`; run both. At what n does `solve` itself lose digits?

A matrix is a verb, not a table.

Next: **Eigendecomposition** — the directions a matrix cannot turn, and how Google used one matrix, applied over and over, to rank the entire web.